AFRILANG Stdlib

std/c/netroute

Bibliothèque AFRILANG `std/c/netroute` (niveau complex, catégorie complex). Elle expose 7 fonction(s) : routLen, routUpp

`import "std/c/netroute"` · `use netroute`

- `routLen(s text) ? number`
- `routUpper(s text) ? text`
- `routLower(s text) ? text`
- `routTrim(s text) ? text`
- `routSplit(s text, delim text) ? list text`
- `routJoin(parts list text, delim text) ? text`
- `routReplace(s text, from text, to text) ? text`

Category: complex | Tier: complex | Functions: 7

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/netroute` (niveau complex, catégorie complex). Elle expose 7 fonction(s) : routLen, routUpp

`import "std/c/netroute"` · `use netroute`

- `routLen(s text) ? number`
- `routUpper(s text) ? text`
- `routLower(s text) ? text`
- `routTrim(s text) ? text`
- `routSplit(s text, delim text) ? list text`
- `routJoin(parts list text, delim text) ? text`
- `routReplace(s text, from text, to text) ? text`

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/netroute"
use netroute
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? routLen(s text) ? number
? routUpper(s text) ? text
? routLower(s text) ? text
? routTrim(s text) ? text
? routSplit(s text, delim text) ? list
? routJoin(parts list text, delim text) ? text
? routReplace(s text, from text, to text) ? text
```

4. Exemples d'utilisation

```
import "std/c/netroute"
use netroute

create result = routLen(/* args */)
say result

create result = routUpper(/* args */)
say result

create result = routLower(/* args */)
say result

create result = routTrim(/* args */)
say result

create result = routSplit(/* args */)
say result

create result = routJoin(/* args */)
say result
```

5. Bonnes pratiques

```
? Préférez `import "std/c/netroute"` en tête de fichier.
? Lancez `afrilang check` avant de livrer.
? Combinez avec les modules stdlib de la même catégorie.
? Voir /stdlib/ pour le source live et les démos playground.
? Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe ? source

```
module netroute
```

```
function routLen(s text) returns number
end

function routUpper(s text) returns text
end

function routLower(s text) returns text
end

function routTrim(s text) returns text
end

function routSplit(s text, delim text) returns list text
end

function routJoin(parts list text, delim text) returns text
end

function routReplace(s text, from text, to text) returns text
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/netroute` (tier complex, category complex). Exports 7 function(s): routLen, routUpper, routLower

`import "std/c/netroute"` · `use netroute`

- `routLen(s text) ? number`
- `routUpper(s text) ? text`
- `routLower(s text) ? text`
- `routTrim(s text) ? text`
- `routSplit(s text, delim text) ? list text`
- `routJoin(parts list text, delim text) ? text`
- `routReplace(s text, from text, to text) ? text`

2. Installation & import

Add the module to your program:

```
import "std/c/netroute"
use netroute
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? routLen(s text) ? number
? routUpper(s text) ? text
? routLower(s text) ? text
? routTrim(s text) ? text
? routSplit(s text, delim text) ? list
? routJoin(parts list text, delim text) ? text
? routReplace(s text, from text, to text) ? text
```

4. Usage examples

```
import "std/c/netroute"
use netroute

create result = routLen(/* args */)
say result

create result = routUpper(/* args */)
say result

create result = routLower(/* args */)
say result

create result = routTrim(/* args */)
say result

create result = routSplit(/* args */)
say result

create result = routJoin(/* args */)
say result
```

5. Best practices

```
? Prefer `import "std/c/netroute"` at the top of your file.
? Run `afrilang check` before shipping.
? Combine with related stdlib modules from the same category.
? See /stdlib/ for the live source and playground demos.
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module netroute
```

```
function routLen(s text) returns number
end

function routUpper(s text) returns text
end

function routLower(s text) returns text
end

function routTrim(s text) returns text
end

function routSplit(s text, delim text) returns list text
end

function routJoin(parts list text, delim text) returns text
end

function routReplace(s text, from text, to text) returns text
end
end
```